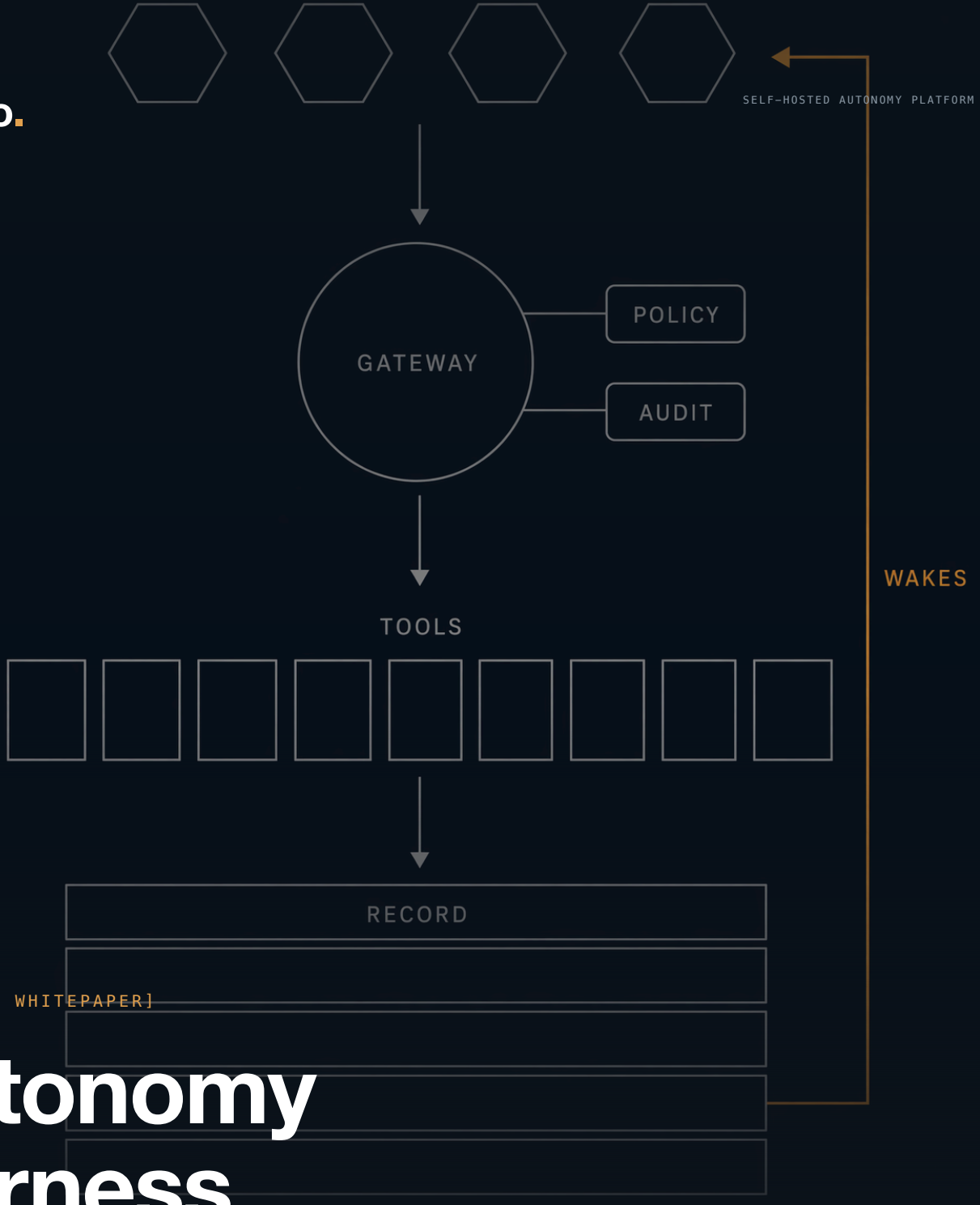
veoveo.



[TECHNICAL WHITEPAPER]

Autonomy Harness

A reactive and proactive cognition stack: autonomous agents that run live operations — on infrastructure you own.

VERSION 1.1 · JULY 2026

[TRANSCRIPT — A HYPOTHETICAL MISSION]

LIVE TRAFFIC WORLD · ECLIPSE SUMO, SIMULATION OF URBAN MOBILITY · LUXEMBOURG SCENARIO

06:41:02	wake	timer · hourly review
06:41:03	context	slice: 2 active missions · 214 vehicles tracked · signals nominal
06:41:04	episode	#4,112 closed — no action required · sleeping

> OPERATOR → PILOT
>
> New mission: survey the northern district for congestion buildup between 07:00 and 09:00, and keep it moving. Boundary attached (survey-north.kml); intersection priorities from ops (priorities.xlsx). Interventions stay inside city signal policy. Report with evidence.

06:42:07	wake	operator message · 2 attachments
06:42:08	reason	two formats I have never seen from this operator — a .kml boundary, an .xlsx priority sheet; I can structure both
06:42:09	act	**duckdb__ingest** priorities.xlsx → ops_priorities · 14 intersections ranked
06:42:10	act	**map__validate_geofence** survey-north.kml → WGS84 polygon · 8.2 km²
06:42:11	act	**memory_write** missions ← 'survey-north' · objectives, boundary, constraints
06:42:12	reason	evidence means cameras and counts — four streams cover the district
06:42:13	act	**subscribe** sumo://congestion · scoped to the boundary
06:42:14	act	**schedule** wake 07:00 · survey window opens
06:42:15	episode	#4,113 closed — the mission is now part of the **world model** · sleeping
07:00:00	wake	timer · survey window open
07:00:03	act	**query_recording** latest-at → 61 vehicles in boundary · edge N-14 densest
07:00:05	act	**perception__analyze_recording** /world/camera/n14 · task t-2211 · sleeping
07:11:40	wake	task result · 1,204 detections · 87 tracks · northbound queue +9%/min
07:11:43	act	**duckdb__ingest** detections → demand per approach
07:11:44	act	**optimization__plan** · minimize delay · options: ops_priorities × demand
07:11:52	reason	plan retimes N-14 and N-9 inside policy; leaves N-2 alone — low priority, low demand; the numbers say so
07:11:53	act	**sumo__set_signal_phase** N-14 N-9 · run_batch(500) · task t-2212 · sleeping
07:19:26	wake	task result · queue at N-14 down **23%** · writing report
07:19:31	report	artifact sealed — counts, plan, clip, rationale · shareable under policy

---

An illustrative transcript. Every surface in it — the wakes, the tools, the durable tasks, the artifacts — is the shipped system. The rest of this paper is the machinery that makes this ordinary.

Abstract

What an operation needs from autonomy is harder to supply than intelligence. Decisions have to follow from what was actually observed, and they have to stay aligned with the mission — with its objectives and constraints, and with the intent behind them when circumstances force a change of plan. Permission has to hold even when the model's judgment fails; especially then. Resilience has to be engineered: processes die, machines are lost, tasks are cut off mid-flight, and the system must come back knowing what it was doing, with the consequences of every interruption defined in advance. What the autonomy is doing has to be observable while it is doing it. And the operator, who answers for all of it, needs assurance that behavior stays within what was approved — and an account of every action, attributable and replayable from the record.

The **Autonomy Harness** is a self-hosted cognition stack built to supply these guarantees structurally. Its foundation is a **world model**: everything the operation senses, remembers, and decides, fused into a durable body of state that agents query rather than accumulate. The mission is part of that model — objectives, constraints, and current beliefs held as data — so intent is something an agent reasons over, and a deviation is measured against the goal it serves before it is adopted and recorded. Reasoning runs in bounded episodes over fresh slices of this model, which is why an agent's knowledge outlives any single process or context window. Between every agent and every tool sits one gateway, enforcing the operator's policy outside the model's reasoning; a refusal there leaves the same evidence an approval does. Work too long for an episode becomes a durable task whose recovery behavior is declared before it starts. Beneath all of this, the platform persists every signal before anything may consume it — losslessly, near a quarter-million messages per second — so the record that grounds a decision can also replay it. New hosted servers attach through a full MCP contract, in Rust or Python, and inherit the task, artifact, identity, policy, and audit guarantees on arrival. A live traffic world exercises the complete loop, and the identical stack runs on a single edge host or inside an air-gapped cluster.

KEYWORDS autonomous agents · world model · runtime governance · durable execution · persist-first recording · edge and air-gap deployment · MCP

> [READING GUIDE]
>
> Section 1 states the contributions; the abstract carries the requirements. Sections 2 through 8 develop the architecture. Section 9 validates the loop on a live traffic world; Section 10 positions the platform as self-improvement primitives; Section 11 covers deployment. Appendix A defines every component.

Contents

1. Contributions

The Autonomy Harness makes six architectural contributions. Each is developed in the section noted and exercised end to end by the validation in Section 9.

1 **The world model as agent context (§3).** Everything the operation senses, remembers, and decides fuses into one durable, queryable model of reality — and the mission lives in it as data, so intent is something an agent reasons over rather than a prompt it was once shown. Episodes draw fresh slices by query; knowledge survives restarts and outlives context windows.

2 **Cognition governed from outside the model (§5).** Every action clears one authenticated gateway and answers to installation policy — refused by default when in doubt. A model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope; even refusals leave evidence.

3 **Persist-first recording (§8).** The ingest socket and the durable writer are one process: every message is written before anything else happens to it. Live files stay readable mid-write, segments are verified before replacement, and capture is lossless at approximately 225,000 messages per second.

4 **Durable tasks with declared recovery semantics (§7).** Long work detaches from the episode that started it and survives process death. Every operation declares its recovery class — resumable, webhook-bound, or fail-closed — and completion wakes the sleeping agent within a second.

5 **A modular capability plane under one contract (§6).** Fourteen hosted server identities ship in the canonical control plane. Any local, bridged, or remote MCP server registers in the validated catalog and immediately answers to the same task, artifact, identity, policy, and audit contracts; domain administration follows the same governed path.

6 **Deployment-invariant cognition (§11).** The same platform, the same agents, and the same operational intelligence run on one edge host, across a cluster, sealed inside an air gap, or hybrid — with provenance included and connectivity on the operator's terms.

2. System Model

The system closes one loop. The world streams into the record; agents perceive from it, decide, and act through governed tools; effects land back in the world, and every step deposits evidence. The shape is Boyd's OODA loop — observe, orient, decide, act — with orientation held in durable state rather than inside the agent. Data moves along this spine in three ways. Producers push inward and agents query back out — observation. Tool calls cross the gateway, and the long ones become durable tasks whose completion wakes a sleeping agent. Whatever remains is the account: authentication decisions, task transitions, artifact grants, and every agent step, landing in the platform store and the decision log.

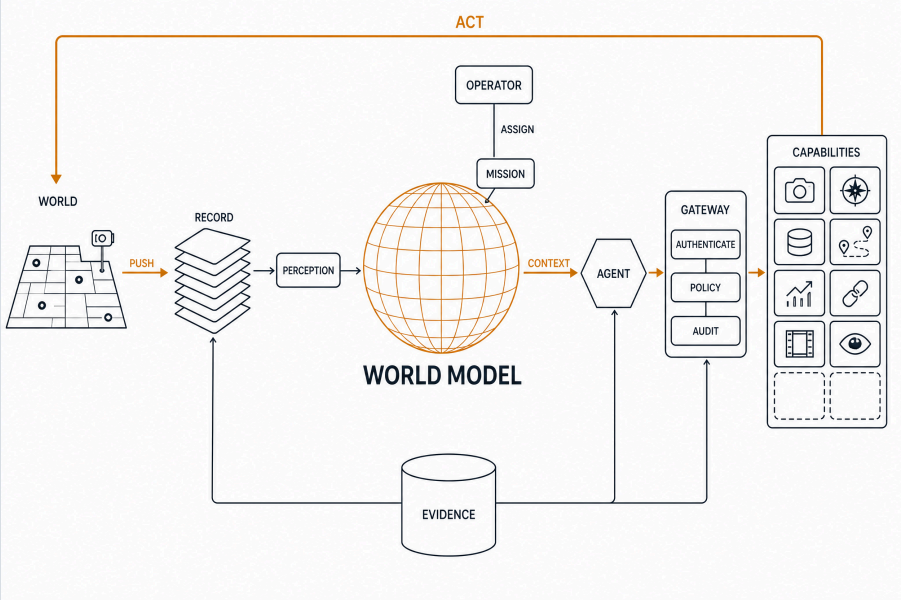


FIGURE 1 — THE AUTONOMY HARNESS, COMPLETE: THE WORLD IS RECORDED AND MODELED, THE MISSION IS ASSIGNED INTO THE MODEL, AGENTS DRAW CONTEXT AND ACT THROUGH THE GOVERNED GATEWAY INTO CAPABILITIES, AND EVERY STEP LANDS IN EVIDENCE.

Flow	Path	Guarantee
Observe	Producers push Rerun log streams to one ingest point; segments become governed catalog records; agents read slices through the recording server.	The hub persists first; a crash mid-write leaves readable files.
Act	Every tool call enters through the gateway under one flat namespace; quick tools answer inline; long tools return a durable task handle immediately.	Task completion pushes a wake; the next episode assembles with the result.
Account	Auth, policy, admin, and task transitions write audit evidence; every decision records what the agent saw, chose, and why.	Usage records meter provider spend and tool activity per principal.

3. The World Model

Agent context is a **world model**: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality. Sensors and simulators stream the physical world into the record; agents keep their beliefs, missions, and constraints in analytical memory; every decision lands in the log — and each act an agent takes becomes part of the model the next episode reasons from. Context is assembled fresh from the world model every episode, so an agent's knowledge survives any restart and outlives any context window. In OODA terms, the world model is the Orient stage made durable and queryable.

The mission is part of the model, held as data rather than prompt text. Objectives, constraints, and current beliefs occupy tables the agent reads and writes through guarded instruments, which makes intent something it can reason about: a proposed deviation gets measured against the objective it serves, adopted deliberately, and recorded with its rationale. Adaptation stays subordinate to the mission because both live in the same queryable state.

A mission does not have to arrive in any particular format. Whatever the source produces — a scenario file from another autonomy stack, an exported plan, an annotated map, a plain document — the agent reads it, interrogates it, and structures it into the model: objectives and constraints become rows in its tables, geometry passes through the Map and Frames servers into real coordinates, and the original lands in the artifact plane with its provenance intact. Integration with an existing stack begins with handing the agent its files.

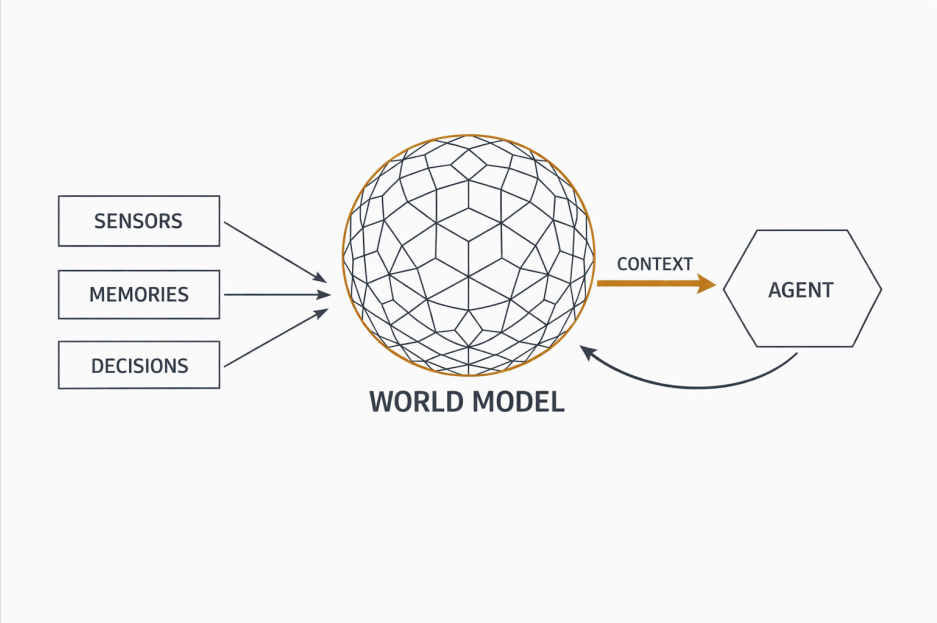


FIGURE 2 — SENSORS, MEMORIES, AND DECISIONS FUSE INTO THE WORLD MODEL; EVERY EPISODE'S CONTEXT IS A FRESH SLICE OF IT.

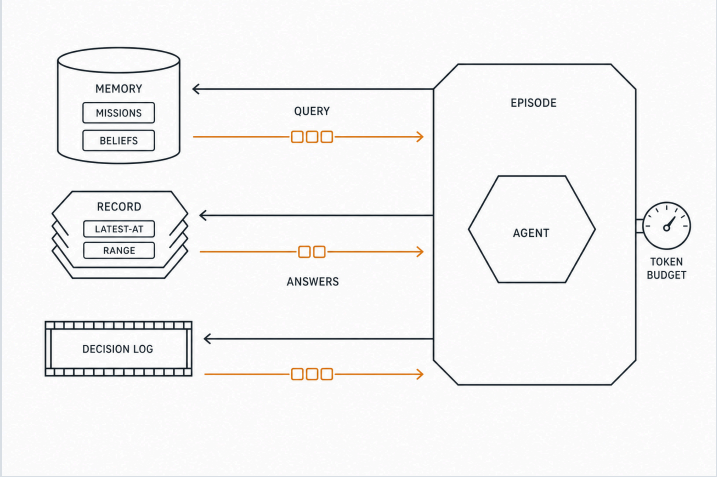
Source	What it contributes	What agents draw
Sensors & sims	The physical world, streamed into the durable time-and-space record.	The world now (latest-at) · how it got here (trajectories and ranges).
Agent memory	Beliefs, missions, constraints, and state in each agent's analytical memory.	What the agent believes and intends — queryable with guarded SQL.
Decision log	Every turn, tool call, and rationale, time-indexed and replayable.	What happened and why — the model of the agents themselves.

3.1 Context, Queried

The world model changes what an episode's context is made of. Context here is assembled by query: the kernel composes each episode from manifest-defined slices, and the agent holds the same instruments, so mid-episode it asks for exactly what it needs — rows from its own memory, a snapshot or trajectory from the record, analytics over anything it has ingested.

Instrument	Example	What the episode gets
`memory_query`	`SELECT id, eta, risk FROM missions WHERE status = 'active'`	Exactly the active missions, as rows; beliefs and constraints join the same way.
`query_recording · latest-at`	every vehicle's position, now	A world snapshot, measured — the state of reality in one call.
`query_recording · range`	vehicle 214, last five minutes	One trajectory, on the original timeline.
`timeline_query`	my tool calls and outcomes, last six hours	The agent reads its own conduct as data.
`duckdb__query`	`SELECT class, count(*) FROM detections GROUP BY window_5m`	Analytics over anything ingested — including perception output.

The strategy carries for two reasons. Tokens spend on answers rather than history, so a mission's entire history compresses to the rows this episode needs. And because every instrument reads the same governed planes, results join — detections land in SQL, SQL rows feed planning, committed plans write back to memory. Section 6.1 shows the composition end to end.



THE EPISODE QUERIES THE PLANES — MEMORY, RECORD, DECISION LOG — AND ONLY ANSWERS COME BACK TO SPEND TOKENS, UNDER A HARD BUDGET.

4. The Agent Kernel

An agent runs indefinitely because durable stores are the memory: every episode's context is assembled fresh from the world model, and everything in flight survives process death. The kernel validates every manifest against its schema before execution. An agent type combines a manifest, a preamble, and SQL migrations. Fielding another agent is an act of configuration, and one installation runs one agent or a team of them.

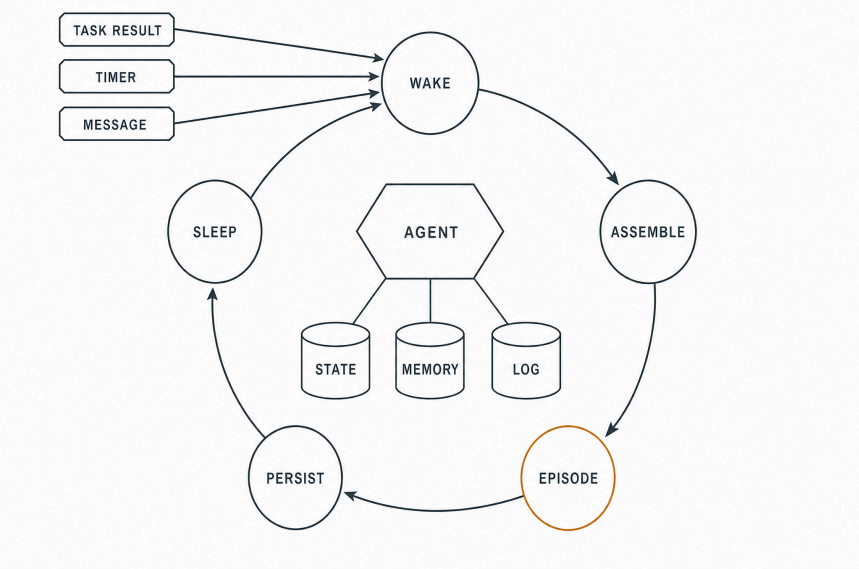


FIGURE 3 — THE LIFECYCLE: WAKE → ASSEMBLE → EPISODE → PERSIST → SLEEP, GROUNDED ON THE DURABLE PLANES.

Stage	What happens
Wake	A batch of wakes, coalesced and deduped; every wake is persisted with its disposition, so the ledger explains why each episode ran.
Assemble	A fresh slice of the world model — current state, beliefs, and history — plus the wake payload, under a token budget.
Episode	One bounded reasoning run. Quick gateway tools answer inline; long tools dispatch as durable tasks. Hooks record every step and terminate over-budget runs.
Persist	Unresolved task descriptors land in the platform store; the episode closes with usage counters and a summary.
Sleep	Idle on the wake bus. A leased watcher per detached task waits on the result; the heartbeat bounds silence.

Wakes: task result · resource change · timer · operator message · elicitation answer. A wake survives process death — descriptors and watcher leases rehydrate from the platform store on any boot, so kill and restart resumes the same tasks.

4.1 Memory Planes and Durability

Memory planes

- **Scheduling truth.** The platform store owns episodes, wakes, task watchers, and leases; the transactional outbox and changefeed deliver wakes across processes.
- **Analytical memory.** A sandboxed analytical database per agent: domain tables (the Pilot: `targets`, `missions`, `waypoints`, `constraints`, `beliefs`), a kv store, and an episode-log projection for context assembly.
- **Decision log.** An append-only recording: every turn, tool call, task lifecycle, prompt, and summary — time-indexed and watchable live.
- The agent reads all three itself: `memory_query` (guarded SQL), `memory_write` (bounded mutations), and `timeline_query` (dataframe reads over the log).

Durability, measured

- Detach at episode end: a task's descriptor outlives the process; any later boot rehydrates the live handle and keeps waiting.
- Token rotation is make-before-break; task ids are principal-scoped, so continuity holds across reconnects.
- The wake is push, arithmetically: with polling pinned at 60 s and the heartbeat at 600 s, a validation agent slept a task's full ~15 s and woke within one second of completion — verified live with a real model end to end.
- Budget hooks book overruns as `budget_terminated`, a first-class outcome; hourly windows defer non-priority wakes.

> [PRINCIPLE]
>
> Agents are data. At boot, schema validation rejects an invalid manifest. The manifest declares the model endpoint, gateway principal, episode limits, budgets, schedule, context sections, and writable tables. A preamble and domain migrations complete the profile; cross-episode instructions live there or in memory.

5. Governance: Edge, Access, and the Gateway

Every client enters through the installation's own ingress, and every action runs the same gauntlet: authenticate, then policy, then audit. Enforcement lives outside the agent's own reasoning — a model can be wrong, drift, or swallow hostile instructions and still cannot act beyond the envelope the operator set. The only path onward carries a gateway-signed identity, and every hosted server lives on the internal network behind it.

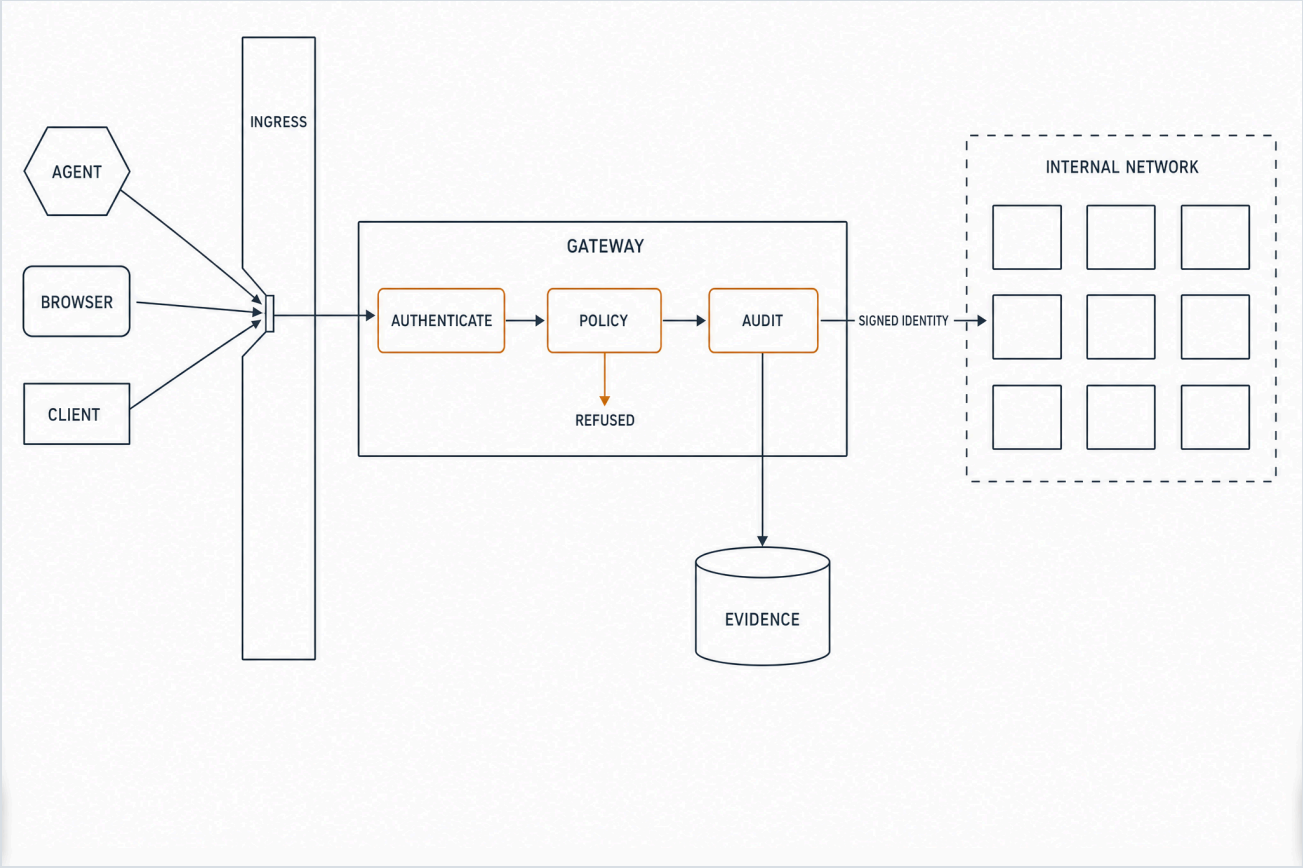


FIGURE 4 — ONE DOOR IN, AUTHENTICATE → POLICY → AUDIT, REFUSED BY DEFAULT, SIGNED IDENTITY ONWARD, EVIDENCE KEPT.

Function	Behavior
Authentication	Browser OAuth with PKCE, client credentials with private-key JWT, identity-assertion grant exchange, JWKS, token issuance and revocation, durable refresh rotation — reuse revokes the family.
Policy & audit	Attribute- and role-based policy over tools, resources, tasks, and artifacts; tenant and data-label checks for regulated data; fail-closed decisions with reason codes; a durable audit trail.
Control plane	Validated server / profile / policy catalog, authenticated live apply and reload, secret references with redaction, an admin API for the console, and policy-checked, audited artifact downloads.
Server administration	`/admin/{profile}/servers/{server}/{path}` resolves the active catalog, authorizes and audits the operation, then forwards a short-lived identity to the owning server's contract-defined `{mount}/admin/*` API. Map uses this path for sources, acquisitions, releases, and mobility profiles.
Protocol surface	Tools, resources and templates, prompts, completions, tasks, subscriptions, notifications, structured content with declared schemas, artifacts, and usage. Tool names gain a prefix only at aggregation; resource URIs retain their owning scheme.

6. The Capability Plane

The canonical control plane defines fourteen hosted server identities. Each server declares the MCP surfaces its domain needs and runs behind gateway-signed identity; a new local, bridged, or remote endpoint registers in the validated catalog and immediately answers to the same task, artifact, identity, policy, and audit contracts. The gateway discovers the declared surface, prefixes tool names only at aggregation, and preserves each resource URI scheme. A client connects once and reaches every capability; a system joins once and reaches every client.

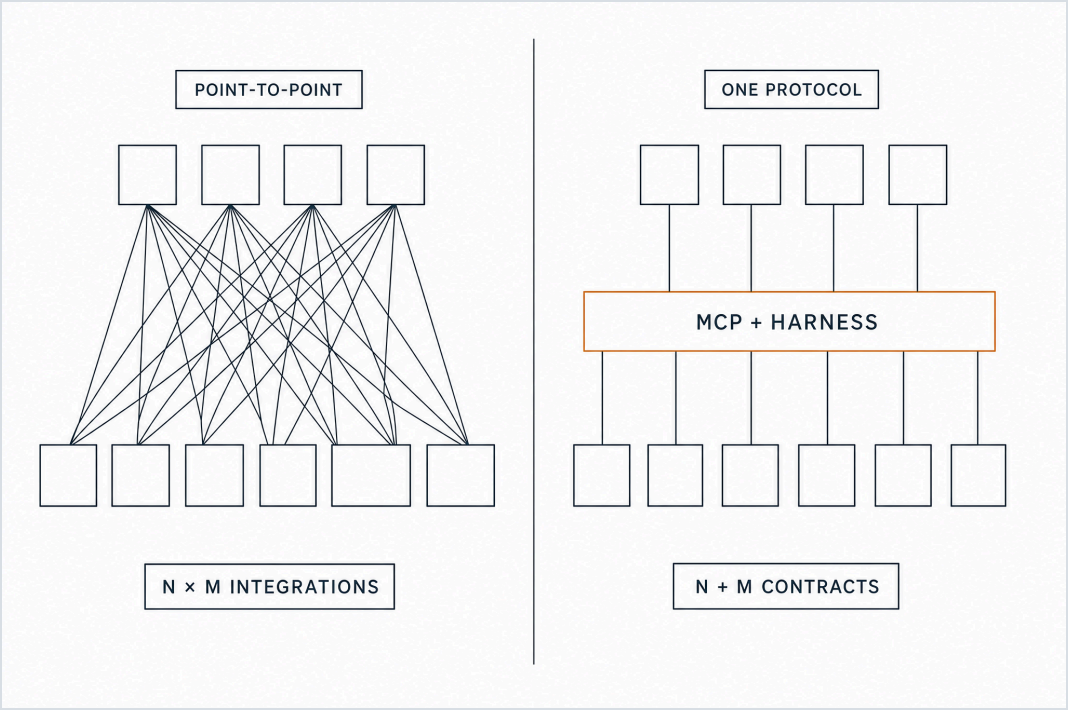


FIGURE 5 — FROM N × M INTEGRATIONS TO N + M CONTRACTS.

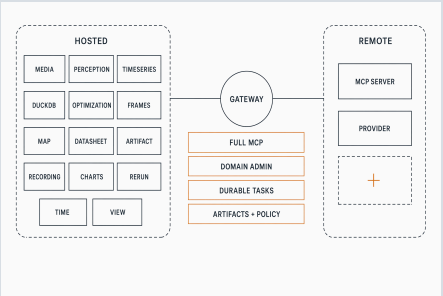


FIGURE 6 — FOURTEEN HOSTED SERVERS AND REMOTE ENDPOINTS; FULL MCP, CONTRACT-DEFINED ADMINISTRATION, DURABLE TASKS, ARTIFACTS, AND POLICY REMAIN ONE GOVERNED SYSTEM.

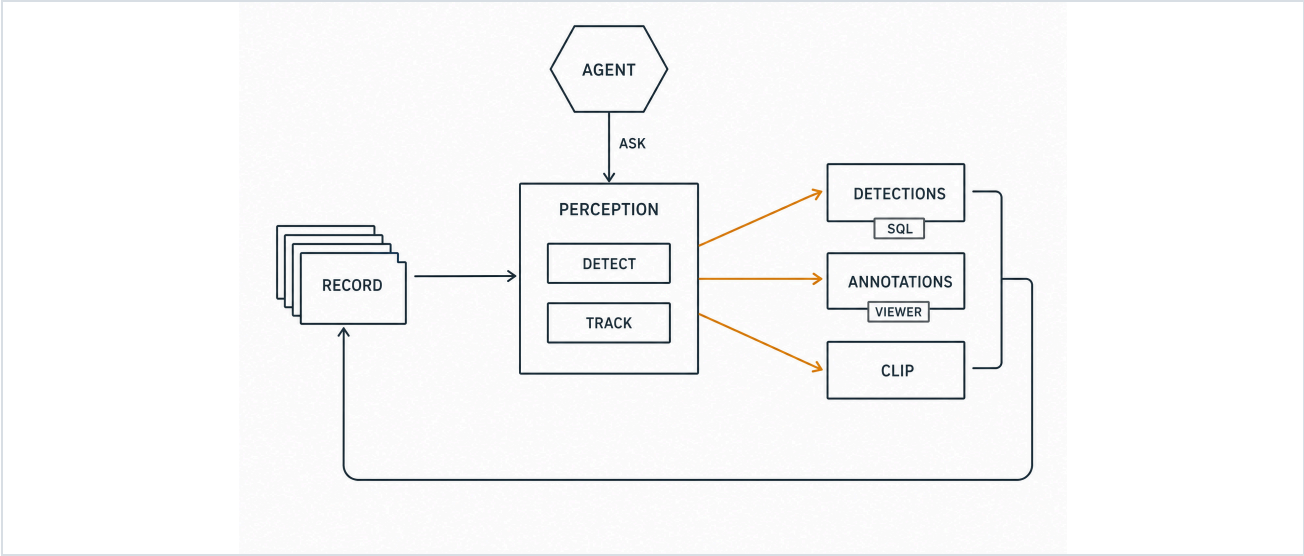
Server	Capability	Representative tools
Media `media`	Provider-backed generative media with webhook-driven completion, a model catalog, and pricing.	`run · models · model_schema · artifact`
Perception `perception`	Bounded GPU inference over the record's video: detection and tracking against governed recording identities, executed locally.	`analyze_recording · extract_clip`
Forecasting `timeseries`	Forecasting over any SQL-readable time-series source, visualized as recordings.	`forecast`
Time `time`	Authority-bound civil, military, GNSS, and mission time; calendars, timelines, clock quality, and temporal events.	`resolve_time · convert_time · assess_clock · evaluate_windows`
SQL `duckdb`	Arbitrary SQL over principal-owned databases inside a locked, resource-bounded sandbox.	`query · execute · ingest · export`
Planning `optimization`	Task-option planning for one or many agents over planning objects or SQL-readable option rows.	`plan`
Frames `frames`	Complete ECEF-rooted frame worlds, immutable revisions, bounded conversion, durable batches, and provenance.	`create_world · publish_world · convert_frame · batch_transform`
Map `map`	Earth geography, governed releases and restrictions, Valhalla and governed-network routes, matrices, and reachable areas with pinned provenance.	`search_locations · inspect_location · route · route_matrix · reachable_area · publish_restriction · validate_route`
Datasheet `datasheet`	Python reference server for dataset preview, column statistics, and durable artifact-backed profiling.	`preview_dataset · column_stats · profile_dataset`
Sharing `artifact`	Discovery, metadata, grants, release, and sharing — the governance surface over the byte store.	`metadata · grant_access · revoke_access · set_release_state · create_share_link · revoke_share_link`
Recordings `recording`	Authorized access to the record: discovery, dataframe queries, subscriptions, publication.	`query_recording · seal_recording`
Charts `charts`	Chart rendering projected through the gateway, turning query results into shareable visuals.	`render_chart · compile_chart · validate_chart · list_chart_types · create_chart_view`
Viewer `rerun`	A live viewer driven entirely over MCP — session, capture, inspection, and UI automation.	`17 interactive tools (connect … batch)`
3D View `view`	Governed static scene compositions with exact inputs and bounded overlays, rendered offscreen on NVIDIA Vulkan.	`create_scene_composition · create_view · set_camera · capture_frame · close_view`

6.1 In Depth: Perception

Every camera the operation records becomes something an agent can question. An analysis runs a detection or detection-plus-tracking model over a chosen stretch of recorded video and returns what was in it: which objects, of which class, where in the frame, when — and, with tracking, which detections are the same object over time. The kinds of questions it answers:

- How many vehicles, by class, passed intersection E14 between 06:00 and 07:00?
- Did anything enter this zone overnight? Show me the frames.
- Follow track 71 — where did it come from, where did it go?

The answer comes back in three forms, one per consumer: **detections as data** — rows an agent queries and joins immediately; **boxes over the original video**, on its own timeline, for an operator scrubbing in the viewer; and **a clip of just the relevant window**, for handing to someone. Analyses run as durable tasks on the installation's own hardware — recorded video stays inside, and results inherit the source's classification.



THE AGENT ASKS, THE RECORD ANSWERS: DETECTIONS FOR SQL, ANNOTATIONS FOR THE VIEWER, A CLIP FOR SHARING — AND EVERYTHING JOINS THE RECORD.

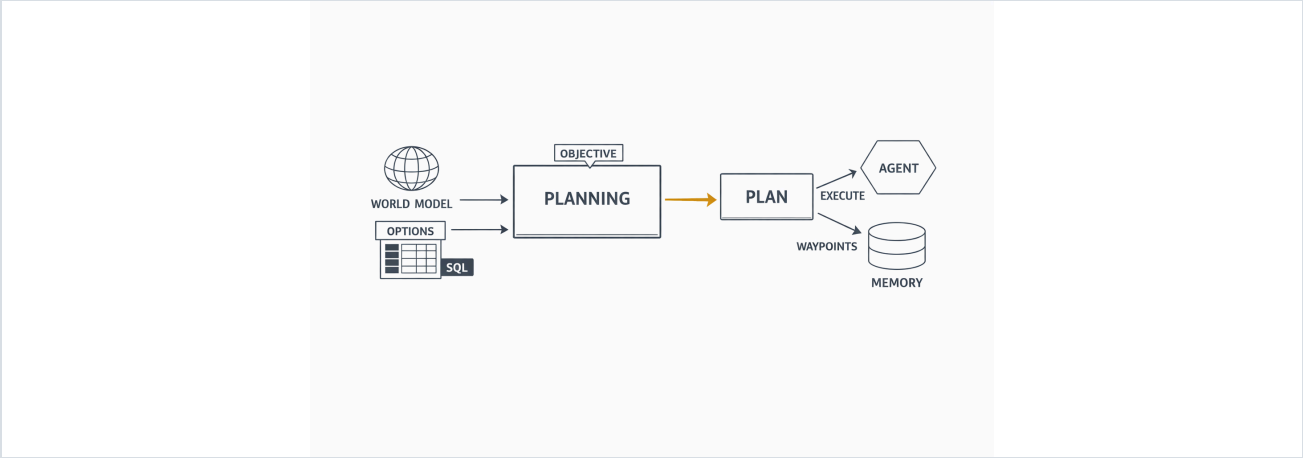
6.2 In Depth: Planning

Planning solves assignment: given agents or assets, the options open to them, and an objective, it returns the best allocation — who does what, in what order, at what cost. The kinds of questions it answers:

- Three survey drones, forty waypoints, battery limits — which drone flies which route?
- Given measured demand per approach, which signal-timing plan minimizes total delay?
- Ten targets, two sensors, shifting priorities — what gets watched this hour?

Options are rows: any SQL-readable table is a valid option set — the mission tables an agent already keeps, or the demand counts perception produced minutes ago. The result is a structured plan the agent executes tool by tool, plus a visualization an operator can inspect before it commits; committed waypoints land back in agent memory.

The request compiles to an exact optimization: one binary decision per option, a linear objective weighing priority, confidence, cost, risk, duration, and resource use under operator-set weights, and constraints covering task requirements, agent concurrency, resource ceilings, dependencies, exclusions, and time-window conflicts between options that share an agent. An embedded solver settles the model in-process, which is why the identical planner runs on an edge host or behind an air gap. The answer is optimal under the declared objective or explicitly infeasible, and it returns with the scores and constraint counts that decided it — the same inputs produce the same plan, and the plan can explain itself.



THE WORLD MODEL AND SQL-READABLE OPTIONS GO IN; THE PLAN COMES OUT — EXECUTED BY THE AGENT, WRITTEN BACK TO MEMORY.

One governed loop

#	Call	Result
1	`wake — resources/updated: sumo://congestion`	A jam forming is a wake, the moment it happens.
2	`query_recording · latest-at`	World snapshot: edge E14 saturated.
3	`perception__analyze_recording /world/camera/e14 · 10 min`	1,204 detections · 87 tracks — cars, buses, trucks, counted per approach.
4	`duckdb__ingest → query`	Demand per approach, as SQL rows.
5	`optimization__plan · minimize total delay · options from SQL`	The winning signal-timing plan, as structured data.
6	`sumo__set_signal_phase · run_batch(500)`	Applied; the agent sleeps through the batch.
7	`wake — task result`	Delay –23%; every step already in the record.

7. Durable Work and Shared Planes

Durable tasks

Every long-running tool runs on the shared task runtime. Task identities are time-ordered; creation, leases, transitions, results, and outbox events are atomic in the platform store. Recovery is declared per operation, so an interruption always has defined, recorded semantics:

Class	Meaning
`resume`	Deterministic, side-effect-safe work may be reclaimed after its lease expires.
`webhook_wait`	A durably submitted provider job waits for its signed webhook. Completion is webhook-only.
`interrupted_indeterminate`	Interrupted mutating work fails closed and is never replayed automatically.

Shared planes

- **Platform store.** One durable coordination authority for identity, policy, control-plane revisions, tasks, artifacts, recordings, map catalogs and releases, agents, wakes, audit evidence, and usage. The transactional outbox and replayable changefeed carry all cross-process work.
- **Artifact plane.** A single byte-level policy-enforcement point: object storage for bytes, the platform store for grants, labels, and release state. Fresh opaque identity per occurrence; hashes provide integrity and tenant-local dedup; artifact references let one server consume another's output.
- **Contract core.** The provider-neutral contract every server shares: Rust newtypes and enums, deployment models, exported JSON Schemas, and gateway-signed identity assertions — all validated fail-closed; servers hold public verification keys only.

> [PRINCIPLE]
>
> Two sharing modes, both explicit: authorized grants to users or groups, still constrained by tenant and label policy; and read-only anyone-with-link bearers for artifacts explicitly marked releasable — expiring, download-limited, and revocable.

8. The Record

Every sensor, estimator, and agent streams into one ingest point, and the hub persists first: each message is written — fsynced — to day-partitioned segment files the moment it arrives. Live files stay readable mid-write, so a crash leaves a decodable record, and every segment is verified before its optimized replacement when it freezes. Every recording and segment becomes a governed record in the platform store, and the recording server answers authorized queries over the same bytes. The record is simultaneously the agents' ground truth and the operation's evidence chain: any run replays, any moment is queryable, and when someone asks why the system acted, the record answers.

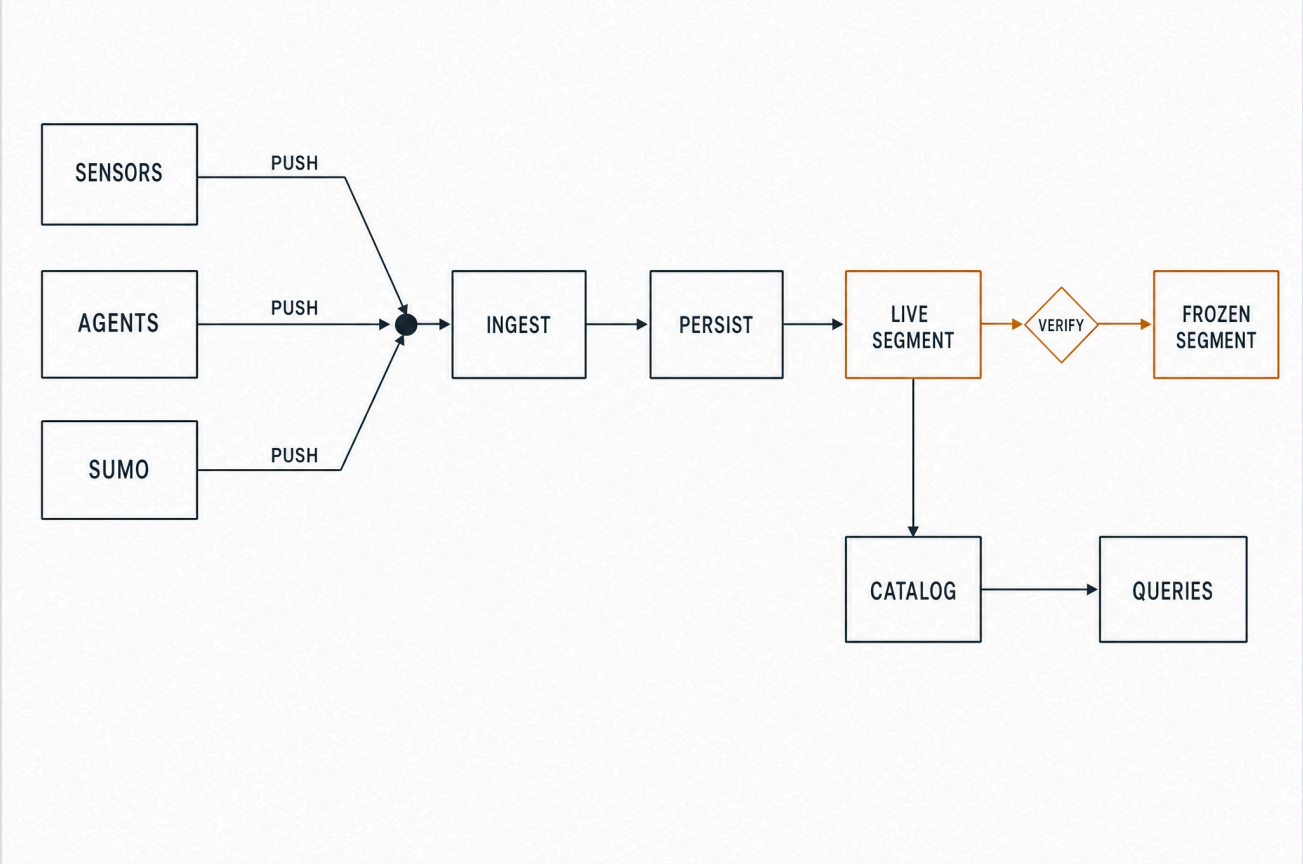


FIGURE 7 — DURABLE CAPTURE: A CRASH LEAVES A READABLE RECORD · A RESTART NEVER OVERWRITES · ~225K MSGS/S LOSSLESS.

9. Empirical Validation

The claims above are exercised on a live traffic world: SUMO — Simulation of Urban Mobility, an open-source traffic microsimulator — runs the pinned, MIT-licensed LuST Luxembourg scenario in its own container; one `sumo-mcp` server owns the single serialized control connection, pushes the world into the record as Rerun streams, and exposes traffic control as governed tools. The long operations are durable tasks the agent detaches from and wakes on — the same sleep/wake the kernel runs everywhere, now driven by a real simulator.

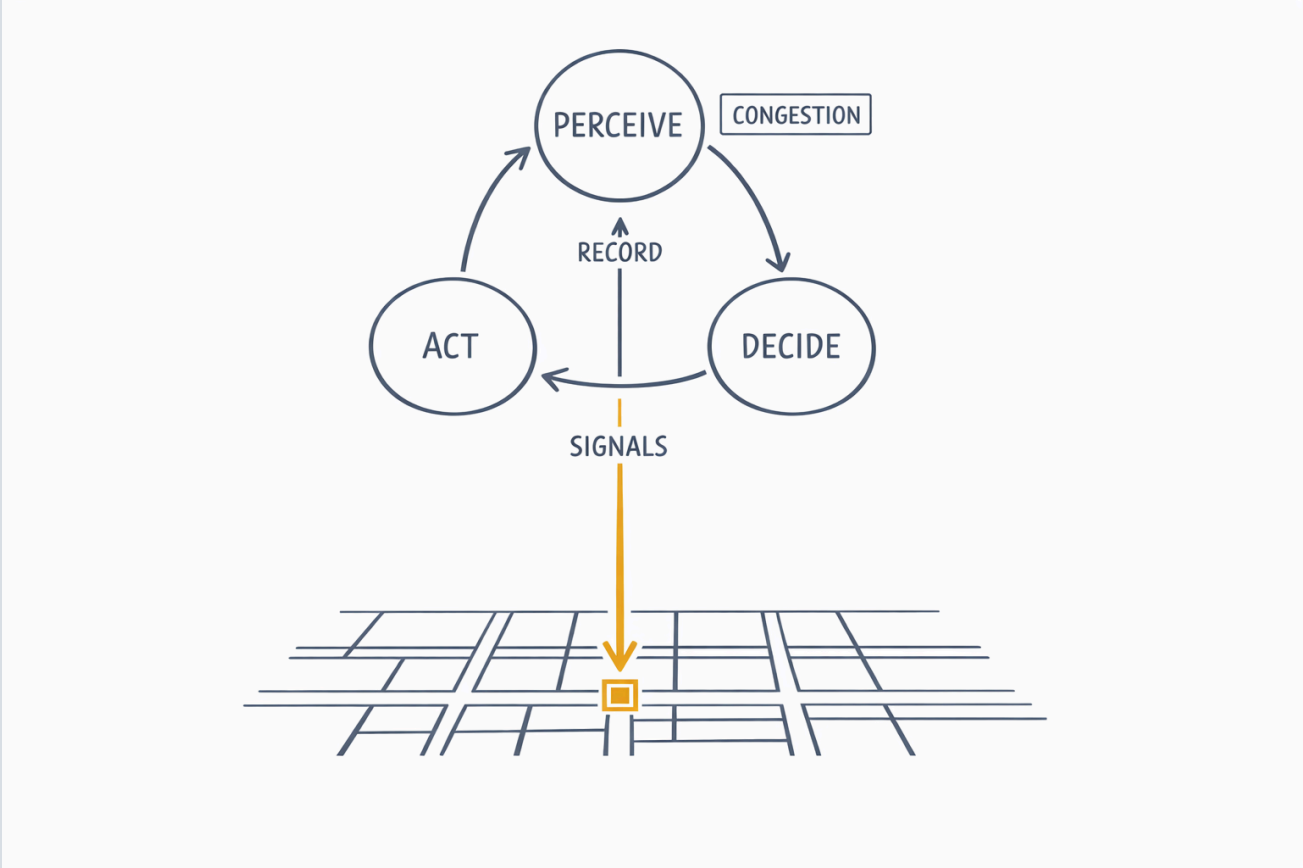


FIGURE 8 — THE CLOSED LOOP ON A LIVE CITY: PERCEIVE FROM THE RECORD, DECIDE, ACT ON SIGNALS AND ROUTES.

Surface	Detail
Sync tools	`query_state` · `describe_scenario` · `set_signal_phase` · `reroute_vehicle` · `set_edge_speed` · `close_lane` · `open_lane`
Durable tasks	`run_batch` (live sim — fails closed on interruption) · `generate_network` · `compute_routes` · `optimize_signals` (real SUMO programs, resumable)
Events	`sumo://congestion` is subscribable — a jam forming is a wake the moment it happens.
Outputs	Successful task outputs upload through a task-bound write capability to the shared artifact plane.

9.1 The Loop, Measured

The exact detach → sleep → wake path: the agent calls a task-augmented tool, receives a durable handle immediately, persists the descriptor, ends the episode, and sleeps. The server runs on; completion pushes a wake, and a fresh episode assembles with the result.

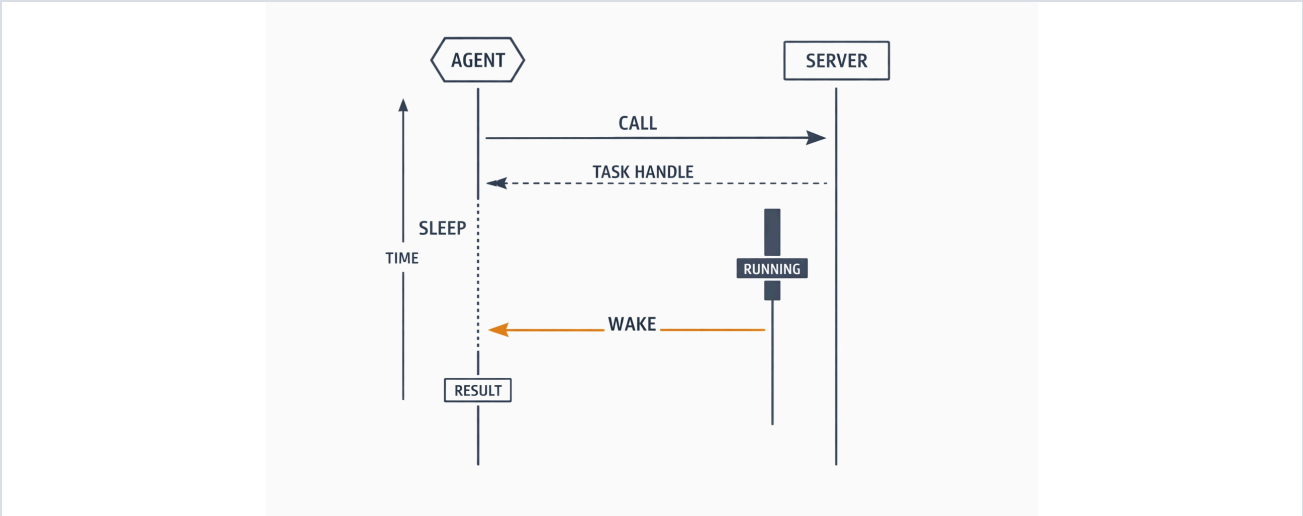


FIGURE 9 — DETACH → SLEEP → WAKE: ZERO CONNECTIONS HELD, ZERO EPISODES BURNED WAITING.

Property	Measured behavior
Capture throughput	~225,000 messages per second, lossless, on the persist-first path.
Crash recovery	kill -9 mid-write leaves a decodable segment; restart writes a sibling and never overwrites; startup reconciles leftovers into catalog records.
Wake latency	With polling pinned at 60 s and the heartbeat at 600 s, the agent slept a ~15 s task in full and woke within one second of completion — push, arithmetically.
Process death	Task descriptors and watcher leases rehydrate on any boot; kill and restart resumes the same tasks with memory intact.

Methodology

- **Smoke harness.** End-to-end scenarios with real process orchestration: fake identity provider, provider, and upstream services; a real enterprise IdP realm; live agent sleep/wake; and the full traffic world in a disposable project. The Map scenario acquires and activates governed fixtures, runs real Valhalla and network routes through the Task API, and verifies that restrictions and release changes invalidate dependent resources.
- **Conformance.** A generic MCP client exercises the full protocol surface, plus auth-metadata discovery and token tooling; continuous integration runs Rust and Python suites, the datasheet reference server, the console build, the IdP integration path, and deployment contract checks on every change.

10. Self-Improvement Primitives

An operation running on the harness produces, as a side effect, exactly what learning consumes. Every episode leaves a complete trajectory — the context the agent saw, the action it took, and what happened next — time-aligned with the world's own record. The harness does not claim the learning program: it does not train models, tune policies, or decide what counts as better. What it supplies is the substrate a program of reinforcement learning, imitation, or scenario-driven improvement is built from, present from the first deployment.

Primitive	What the platform guarantees	What can be built on it
Complete experience	Every episode records its context slice, tool calls, outcomes, and rationale, aligned to the same timeline as the world's sensor record.	Trajectory datasets — state, action, outcome — mined by query; reward signals computed after the fact from measured results.
Derived ground truth	Perception turns recorded video into labeled facts on the source timeline; the record is lossless and replayable.	Training data extracted from operations, with provenance; labels that survive audit.
A world to practice in	The same contracts drive live and simulated worlds; recorded anomalies replay as scenarios.	Policy evaluation and learning rollouts against replayed or simulated reality, off the live system.
Measurable objectives	Missions, constraints, and outcomes live as queryable state; usage and results are metered per principal.	Objective and reward functions defined over data the operation already keeps.
Governed rollout	Agents are data — a candidate variant is a manifest — and every variant acts through the same gateway, budgets, and policy as anything else.	Shadowed or staged variants whose failures are contained by construction, promoted as governed change.
Lineage	Artifacts carry provenance; control-plane revisions and the audit trail record what changed, when, and on whose authority.	A defensible chain from data to training run to deployed variant.

What remains the learning program's to design — the algorithm, the training infrastructure, the criteria for promotion — designs against a platform where experience is already durable, objectives are already measurable, and rollout is already governed.

> [POSITION]
>
> The platform does not train models; it makes experience durable, objectives measurable, and rollout governable. Learning is designed on top — and the primitives it needs are already in place.

11. Deployment Considerations

Deployment is a spectrum, and cognition is constant across it. The same platform runs at the operation's edge on one host, across a cluster, sealed inside an air gap, or hybrid — an edge installation that reaches remote capability when links allow. Wherever it lands, it carries the full stack: one agent or a team of them, every hosted capability server, and the operational intelligence to run the mission — the console, the record, and the audit trail.

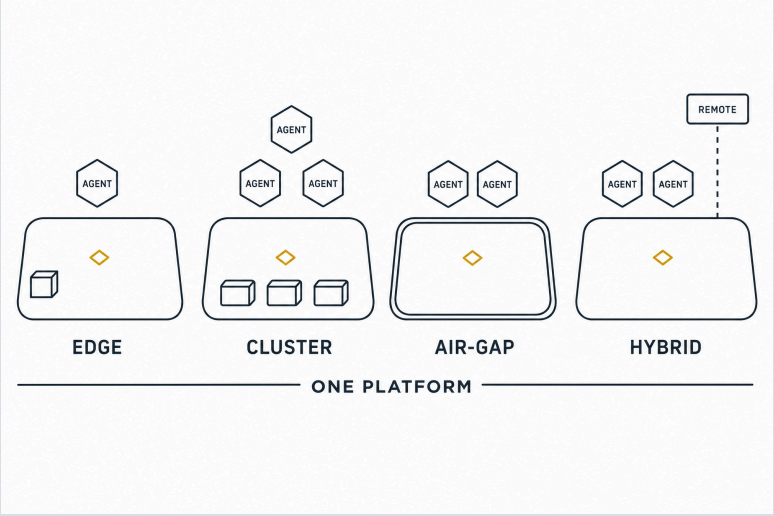


FIGURE 10 — SAME PLATFORM, SAME COGNITION — EDGE, CLUSTER, AIR-GAP, OR HYBRID; ONLY HYBRID REACHES OUT, AND ON THE OPERATOR'S TERMS.

Form	Shape
Edge	One host at the operation: a single command brings up the full stack behind a loopback edge. Public exposure is the operator's ingress decision.
Cluster	The Helm form: one platform-store StatefulSet, default-deny network policy, optional service-mesh mTLS, separate migration and runtime credentials, operator-supplied telemetry and SIEM wiring.
Air-gap	The verified offline bundle: pinned images, installation material, configuration schemas, checksums, and SBOMs. Installation and verification complete entirely inside the boundary.
Hybrid	An edge installation that registers remote MCP endpoints and provider-backed servers. The reach is governed like everything else — same policy, same audit, same task contract.

12. Conclusion

The Autonomy Harness closes the loop that persistent autonomy requires. An agent here reasons from a world model that carries the operation's history and its mission alike. The gateway holds the operator's policy whatever the model believes. Work survives process death because its recovery was declared before it started, and evidence accumulates as a side effect of running rather than as a reporting obligation afterward. A live traffic world exercises all of it, and the platform deploys unchanged from a single edge host to an air-gapped cluster.

The same contracts are the foundation for what comes next. Teams of agents can coordinate over a shared world model; richer hosted servers can register from the ecosystem; guarantees can be expressed over mission outcomes rather than individual actions. Each grows on the spine this paper described.

> [THESIS, RESTATED]
>
> Ground the agent in a world model that carries the operation and its mission. Enforce the rules outside the model. Declare how work fails before it starts, and write the record before anything reads it. One operator owns all of it, anywhere the mission runs.

Appendix A: Component Reference

Component	Role	What it is
ingress	ours	The single public door, owned by the operator. One place to defend and observe.
gateway	ours	The governed protocol boundary: authenticates the caller, checks policy, projects full MCP server surfaces, proxies server administration under declared request and response schemas, and writes the audit trail.
console	ours	The cockpit: the operations UI plus a session backend that owns browser login, encrypted sessions, and CSRF enforcement.
agent kernel	ours	The runtime for long-lived agents: short episodes, durable memory, wake on a result, a timer, or a message.
hosted MCP server	ours	A domain boundary with explicit request and response models, the MCP surfaces it needs, shared task and artifact contracts, and optional administration under its canonical mount.
Map server	ours	Earth geography and logistics routing over governed, versioned releases; its administrative API owns sources, acquisition, activation, rollback, and mobility profiles.
world model	concept	What an agent knows: everything the operation senses, remembers, and decides, fused into one durable, queryable model of reality.
platform store	OSS	SurrealDB — the platform's memory: identity, policy, tasks, artifacts, recordings, map catalogs and releases, agents, audit, and the transactional outbox.
task runtime	ours	The ledger for long work: atomic leases, transitions, results, and declared recovery per operation.
ingest socket	OSS	Where every producer pushes: one gRPC endpoint (the Rerun proxy), rebroadcast live to anyone reading.
persist-first writer	ours	Embeds the ingest socket and fsyncs every message to disk before anything else — the write happens first.
.rrd segment	format	The record on disk: one growing file per dataset / day / recording, readable even mid-write.
catalog	ours	The card catalog: stable installation identity for every recording and segment; startup reconciles crash leftovers.
recording server	ours	The reading room: authorized discovery, latest-at / range / dataframe queries, subscriptions, and sealing to an artifact.
SUMO / TraCI	OSS	The traffic simulator and its control socket — a live world producing positions and states to record and reason about.
sumo-mcp	ours	The bridge: the one owner of the simulator's control socket, pushing the world into the record and exposing governed control.
MCP	protocol	The shared protocol for tools, resources, prompts, completions, tasks, subscriptions, notifications, and structured content with declared schemas; one gateway governs the complete surface.
durable task	concept	A tool call you walk away from: the handle returns immediately; the row, lease, and result live in the platform store.
resource · subscribe	concept	A condition that wakes you: subscribing to a resource means the agent is notified the moment it changes.